

1. What is Problem Solving in AI?

- Problem solving in AI means finding a sequence of actions that leads from an initial state to a goal state.
- We model the task as a search through a space of possible states (the state space).
- Search algorithms systematically explore states to find a solution.

Key Idea: A problem can be solved by searching through a space of possible states.

Example: Find a route from Arad to Bucharest, solve the 8-puzzle, color a map, or find a plan for a robot to reach a target.

3. Problem Formulation

- To apply a search algorithm, we must formulate the problem formally: define states, actions, goal, etc.
- Different formulations of the same real-world task are possible, depending on what information we include in the state and how we define actions.

Example: Route finding

- State: current city (e.g., Arad)
- Actions: drive to a neighboring city
- Goal: reach Bucharest
- Path cost: total distance traveled
- The exact formulation affects the search space size and the efficiency of the search.

2. Problem-Solving Agent Idea

- A problem-solving agent formulates a goal and uses search to find an action sequence to reach it.
- It perceives an initial state, formulates a goal, searches for a solution (a sequence of actions), and then executes the actions.



Main steps:

1. Formulate the problem (states, actions, goal, cost).
2. Search for a solution using a search algorithm.
3. Execute the sequence of actions in the real world.

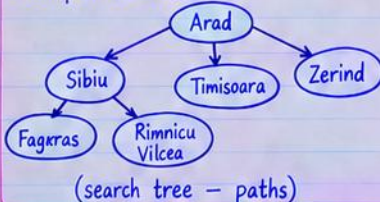
4. Search Problem Components

Initial state	State where the agent starts.
Actions(s)	Set of valid actions available in each state.
Transition model	Resulting state after applying an action. (often given by a successor function)
Goal test	Checks if a state satisfies the goal condition.
Path cost	Cost of a path (e.g., distance, time, number of steps).
State space	Set of all possible states reachable from the initial state.
Solution	A sequence of actions that leads from the initial state to a goal state.
Optimal solution	A solution with the lowest path cost among all solutions.

5. Search Tree vs. State Space

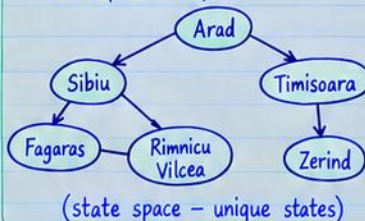
Search Tree

- Nodes are paths (states with action sequences).
- Repeated states can appear many times via different paths.
- Simpler to understand.



State Space (Search Graph)

- Nodes are states (unique).
- Avoids repeated states.
- More efficient (can use an explored set).



Key Idea:

- The search tree contains all possible paths, so states can repeat.
- The state space is a graph of unique states, often with an explored set to avoid revisiting states.
- A solution is a path from the initial state to a goal state.

6. Real-World Examples of Search Problems

Problem	State Space	Actions	Goal Test	Path Cost
Route finding	Locations (cities)	Move to neighboring city	Is destination city?	Distance or time
8-puzzle	Board configurations	Slide a tile (Up, Down, Left, Right)	Is goal configuration?	Number of moves
Map coloring	Color assignments	Assign a color to a region	Are all adjacent regions different colors?	Number of colored regions (or just find any solution)
Robotics (path planning)	Robot configurations	Move, turn, pick, place	Has target been reached?	Distance, time, or energy

Example: 8-Puzzle

Initial state				Goal state		
2	8	3	→	1	2	3
1	6	4		8		4
7		5		7	6	5

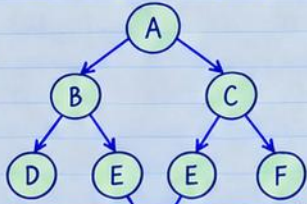
- State: arrangement of tiles
- Actions: move blank (Up, Down, Left, Right)
- Goal test: matches goal configuration
- Path cost: number of moves

“Search is the engine of problem solving. The better we explore, the closer we get to the goal.” — AIMA (Ch.3)

1. Search Tree vs. Search Graph

Search Tree

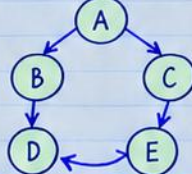
- States can appear multiple times (repeated states allowed).
- Represents all possible sequences of actions from the initial state.
- Simpler to understand.
- Can be very large (many duplicates).



Same state (E) appears twice in the tree.

Search Graph

- Avoids repeated states (keeps only one copy).
- Uses an explored set to detect and avoid repetitions.
- More efficient in time and memory.
- Usually preferred in practice.



Each state appears only once (uses explored set).

2. Frontier and Explored Set

Frontier (OPEN set)



Stores generated but not yet expanded states. Implemented as a queue (BFS) or priority queue (UCS).

Explored Set (CLOSED set)



Stores states that have already been expanded. Used to avoid revisiting states (in graph search).

Key Idea:

Graph search maintains a frontier and an explored set. A state is expanded at most once, which avoids repeated work and makes the search more efficient.

3. General Graph-Search Algorithm (Idea)

Idea:

- Initialize the frontier with the initial state.
- Initialize the explored set to empty.
- Repeat:
 - If frontier is empty, return failure.
 - Remove a state from the frontier according to the search strategy (FIFO, lowest cost, etc.).
 - If the state is a goal, return the solution.
 - Add the state to the explored set.
 - Generate successors and add to frontier if not already in frontier or explored set.

Graph-Search (high level)

```

frontier ← {initial state}
explored ← ∅
while frontier is not empty:
  node ← remove from frontier
  if goal(node) return solution
  add node to explored
  for each successor in successors(node):
    if successor not in frontier
      and not in explored:
      add to frontier
  
```

4. Breadth-First Search (BFS)

- Expands the shallowest unexpanded node first.
- Uses a FIFO queue for the frontier.
- Explores level by level (increasing depth).
- Complete (if branching factor is finite).
- Optimal (if all step costs are equal).
- Time complexity: $O(b^d)$
- Space complexity: $O(b^d)$ (large because keeps all nodes at a given depth).

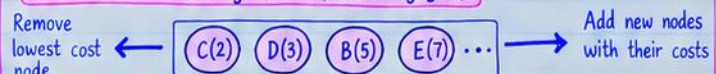
Frontier as a Queue (FIFO)



5. Uniform-Cost Search (UCS)

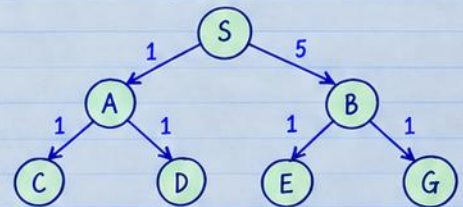
- Expands the node with the lowest path cost $g(n)$ first.
- Uses a priority queue ordered by path cost $g(n)$.
- Complete (if step costs are positive).
- Optimal (finds the least-cost solution).
- Time complexity: $O(b^{1+\lceil C^*/\epsilon \rceil})$ (exponential in solution cost)
- Space complexity: $O(b^{1+\lceil C^*/\epsilon \rceil})$.

Frontier as a Priority Queue (ordered by $g(n)$)



6. Small Example: BFS vs. UCS Expansion Order

Consider the following search tree with step costs (shown on the edges). Goal state is G.



BFS Expansion Order
(step costs ignored)

S, A, B, C, D, E, G
Finds shallowest goal (6 steps in this example).

UCS Expansion Order
(uses path costs)

S, A, C, D, B, E, G
Expands in order of increasing path cost. Finds optimal goal (path cost 6).

7. Comparison of BFS and UCS

Property	Breadth-First Search (BFS)	Uniform-Cost Search (UCS)
Node selection	Shallowest node first (FIFO queue)	Lowest path cost $g(n)$ first (priority queue)
Complete	Yes (if branching factor is finite)	Yes (if step costs are positive)
Optimal	Yes (if all step costs are equal)	Yes (with positive step costs)
Time complexity	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$
Space complexity	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$
When to use	When step costs are uniform or unknown	When step costs are non-uniform and positive

1. Depth-First Search (DFS)

- Explores as deep as possible along each branch before backtracking.
- Uses a stack (LIFO) for the frontier.
- Expands the most recently generated node first.
- Can go deep into the tree and may get stuck in infinite paths (if no repeated-state checking).

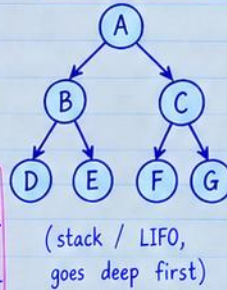
Pros:

- Uses little memory ($O(bm)$).
- Can find a solution quickly if it is deep and to the left.

Cons:

- Not complete (can get stuck in infinite depth).
- Not optimal (may find a non-optimal solution).

Expand order
(A, B, D, E, C, F, G)

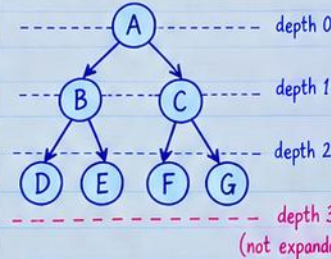


2. Depth-Limited Search (DLS)

- A variant of DFS with a depth limit ℓ .
- Nodes at depth ℓ are not expanded.
- Uses a stack (LIFO), like DFS.
- Avoids infinite paths by limiting the depth.

Key Idea:

Impose a maximum depth limit ℓ to avoid infinite search in deep or infinite state spaces.



Pros:

- Complete if $\ell \geq d$ (solution depth).
- Uses little memory.

Cons:

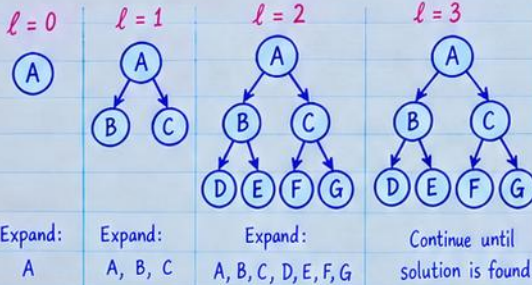
- Not optimal.
- If $\ell < d$, it fails to find a solution.

3. Iterative Deepening Search (IDS)

- Repeatedly applies depth-limited search with increasing depth limits: $\ell = 0, 1, 2, 3, \dots$
- Combines the space efficiency of DFS with the completeness and optimality of BFS (for equal step costs).
- Each iteration is a complete DLS, so it will eventually find a solution if one exists.

Why useful?

- Avoids choosing a wrong depth limit.
- Often finds shallow solutions quickly.
- Good for large or infinite state spaces.



Properties:

- Complete (if b is finite).
- Optimal (if all step costs are equal).
- Time: $O(b^d)$
- Space: $O(bd)$

5. Comparison of Uninformed Search Algorithms

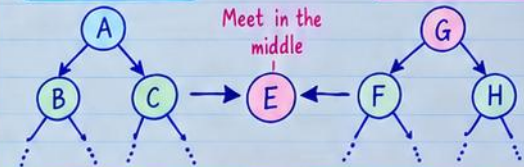
Algorithm	Frontier Data Structure	Complete?	Optimal? (equal costs)	Time Complexity	Space Complexity
BFS	Queue (FIFO)	Yes (if $b < \infty$)	Yes	$O(b^d)$	$O(b^d)$
DFS	Stack (LIFO)	No (infinite paths)	No	$O(b^m)$	$O(bm)$
DLS (limit ℓ)	Stack (LIFO)	Yes if $\ell \geq d$	No	$O(b^\ell)$	$O(b\ell)$
IDS	Repeated DLS	Yes (if $b < \infty$)	Yes	$O(b^d)$	$O(bd)$
Bidirectional Search	Typically BFS (from both ends)	Yes (if both sides finite)	Yes	$O(b^{d/2})$	$O(b^{d/2})$

4. Bidirectional Search

- Search forward from the initial state and backward from the goal state simultaneously.
- Stop when the two searches meet.
- Usually implemented with BFS from both sides.
- Useful when both the initial state and goal state are known, and branching factor is similar in both directions.

Forward search
(from initial state)

Backward search
(from goal state)

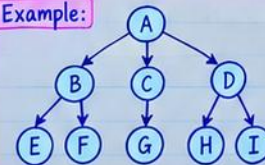


Key Idea: By searching from both ends, the effective search depth is reduced from d to about $d/2$, which can significantly reduce the number of nodes expanded.

6. Branching Factor, Solution Depth, Maximum Depth

- Branching factor (b): maximum number of successors of any node in the search tree.
- Solution depth (d): depth (number of steps) of the shallowest goal node.
- Maximum depth (m): maximum depth of the state space (may be infinite).

Example:



Branching factor $b = 3$
(each node has ≤ 3 children)
Solution depth $d = 2$
(e.g., E, F, G, H, I)
Maximum depth $m = 4$
(in this example)

"Depth-first search is simple and memory efficient, iterative deepening is both complete and optimal, and bidirectional search can be dramatically faster when the goal is known." — AIMA (Ch.3)

1. 8-Puzzle as an Example Search Problem

- 8-puzzle: a 3×3 board with 8 numbered tiles and one blank space.
- An action slides a tile into the blank space.
- Start state, goal state, actions, path, and path cost can be defined, so it is a search problem.
- State space: all $9!/2 = 181,440$ reachable states.
- Branching factor $b \leq 4$ (2, 3 or 4 depending on blank).
- Step cost is usually 1 per move (uniform cost).

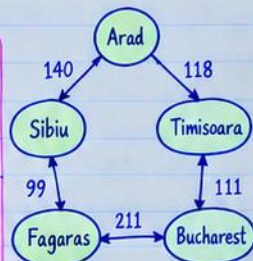


2. Route-Finding and Real-World Examples

- Route finding: find a path between two locations (e.g., cities) on a map.
- States: locations (cities), Actions: move to neighboring cities.
- Path cost: distance, time, fuel cost, etc.
- Many real-world problems can be formulated as search problems.

Examples:

- Navigation (GPS): find shortest or fastest route.
- Game playing: find a sequence of moves to win.
- Robotics: find a sequence of actions to reach a goal.
- Scheduling: assign tasks to achieve a goal state.
- Planning: sequence of actions to achieve objectives.



Example: route finding in Romania map (from AIMA, Ch. 3)

3. Performance Criteria for Search Algorithms

We evaluate search algorithms using the following criteria:

- (a) Completeness**

 - Returns a solution if one exists.
 - Important for guaranteed results.

(b) Optimality

 - Returns a least-cost (optimal) solution.
 - Depends on step costs and algorithm.

(c) Time Complexity

 - Number of nodes generated/expanded.
 - Usually measured as a function of b and d .

(d) Space Complexity

 - Maximum number of nodes stored in memory.
 - Important for large state spaces.

4. Asymptotic Expressions

- We use asymptotic notation to describe time and space complexity in terms of branching factor b and solution depth d .
- Common expressions:

- $O(b^d)$: exponential in depth d (typical for BFS)
- $O(b^m)$: exponential in maximum depth m (typical for DFS)
- $O(d)$: linear in depth (e.g., space for DFS)
- $O(bd)$: linear in b and d (e.g., space for BFS)

b : branching factor (maximum number of successors per node)
 d : depth of the shallowest solution
 m : maximum depth of the state space

5. Comparison of Uninformed Search Strategies

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity	Notes
Breadth-First Search (BFS)	Yes	Yes	$O(b^d)$	$O(b^d)$	Expands shallowest nodes first. Uses a queue.
Depth-First Search (DFS)	No	No	$O(b^m)$	$O(b^m)$	Explores as deep as possible. Uses a stack.
Uniform-Cost Search (UCS)	Yes	Yes	$O(b^{1+\lceil \frac{C^*/\epsilon}{\epsilon} \rceil})$	$O(b^{1+\lceil \frac{C^*/\epsilon}{\epsilon} \rceil})$	Expands lowest path cost first. Uses a priority queue.
Depth-Limited Search (DLS)	No	No	$O(b^l)$	$O(b^l)$	Like DFS, but with depth limit l .
Iterative Deepening Search (IDS)	Yes	Yes	$O(b^d)$	$O(bd)$	Repeated DLS with increasing depth limit. Uses little memory.

6. Chapter Recap / Key Takeaways

- A search problem is defined by state space, initial state, actions, transition model, goal test, and path cost.
- 8-puzzle and route finding are classic examples of search problems.
- Search algorithms are evaluated by completeness, optimality, time complexity, and space complexity.
- Time and space complexity are expressed using asymptotic notation such as $O(b^d)$ and $O(b^m)$.
- BFS and UCS are complete and optimal (with positive costs), but use more memory.
- DFS uses less memory but may not be complete and is not optimal.
- IDS combines the advantages of BFS and DFS.

Key Idea: The choice of search algorithm depends on the problem requirements and the available computational resources (time and memory).

“Different search algorithms make different trade-offs between completeness, optimality, time, and space. The right algorithm depends on the problem.”

— AIMA (Ch.3)